SOFTWARE ENGINEERING · CHAPTER 08

Class Diagrams

Class diagrams model the static structure of a system by showing classes, attributes, operations, and relationships. They support object-oriented analysis, design, and communication.

UML 2.5.1 | Mermaid 11 classDiagram | JDK 25 LTS · Jakarta Persistence 3.2

AGENDA

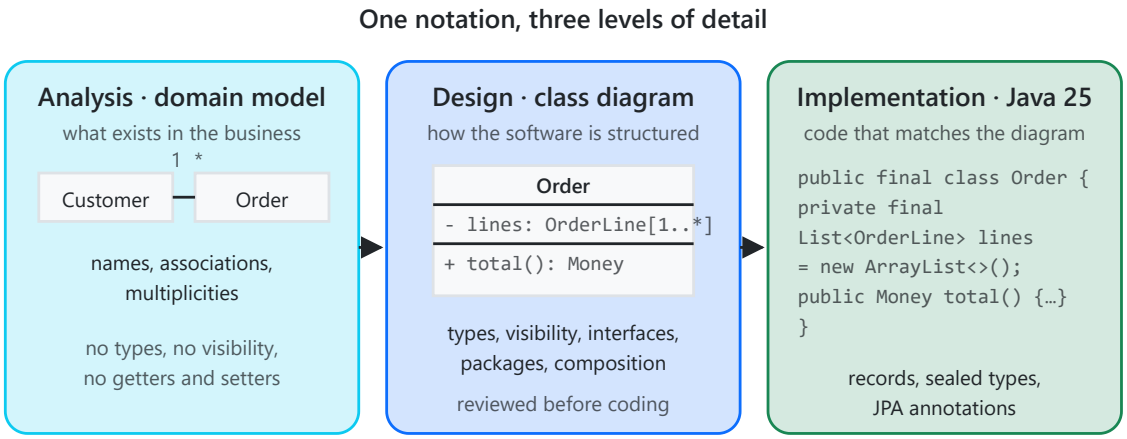
What we will cover

1. Classes, interfaces, attributes, and operations
2. Visibility, types, and responsibilities
3. Associations, navigability, and multiplicity
4. Aggregation and composition
5. Generalization and realization
6. Dependencies and usage relationships
7. Abstract classes and interfaces
8. Packages and namespaces
9. Mapping domain models to implementation
10. Applying SOLID principles to class design
11. Chapter Quiz
12. Lab-08

Click any item to jump directly to that topic.

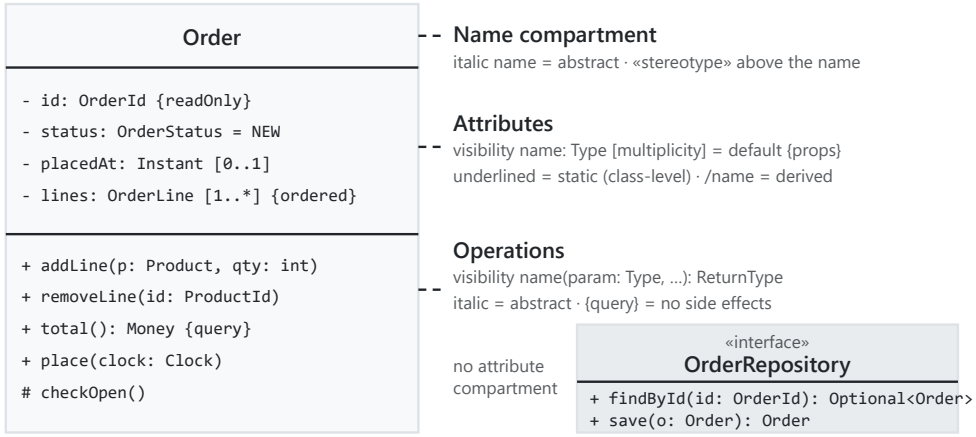
Why class diagrams? structure, not behaviour

- Chapter 07 answered *who wants what* (actors, use cases). A class diagram answers *what exists*: the types, their data and the links between them.
- The **same notation** serves three levels of detail. An analysis *domain model* has names and associations; a design diagram adds types, visibility, interfaces and packages; the implementation is Java code that must match it.
- Running example: the online shop from Chapters 02-06 (`Customer`, `Order`, `OrderLine`, `Product`, `Money`, `Payment`, `PaymentGateway`, `OrderRepository`).
- Tooling: Mermaid 11 `classDiagram` text (versioned with the code) plus inline pictures where Mermaid has no notation.



UML 2.5.1 structure diagram: it shows what exists, not what happens (that is Chapters 09 and 10)

Anatomy of a class: three compartments



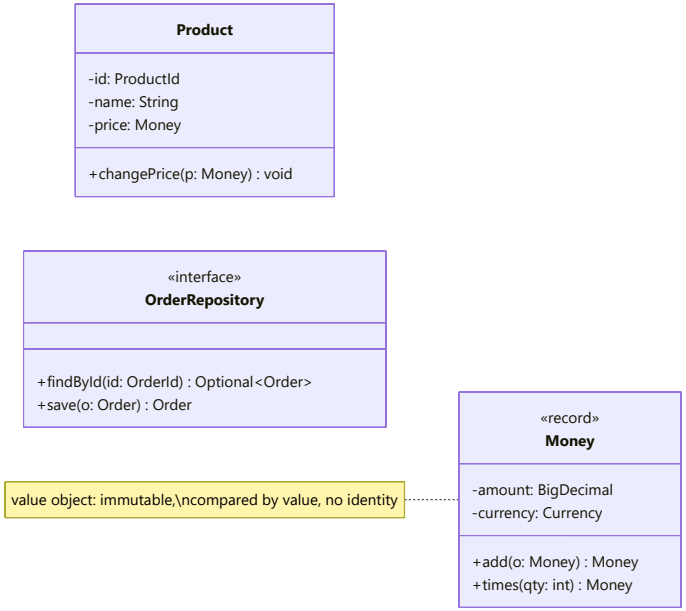
- A **class** is drawn as a rectangle with up to three compartments: name, attributes, operations. Empty compartments may be hidden.
- An **interface** is a class box with the keyword `«interface»`; it declares operations that realizing classes must implement.
- Show only what matters for the reader: a domain model hides types and getters; a design diagram shows the members that carry decisions.

```java
// domain/Order.java  (matches the box on the left)
public final class Order {
    private final OrderId id;
    private OrderStatus status = OrderStatus.NEW;
    private Instant placedAt;
    private final List<OrderLine> lines = new ArrayList<>();

    public Order(OrderId id) { this.id = id; }

    public void addLine(Product p, int qty) {
        checkOpen();
        lines.add(new OrderLine(p.id(), p.price(), qty));
    }
    public void removeLine(ProductId id) {
        checkOpen();
        lines.removeIf(l -> l.productId().equals(id));
    }
    public Money total() {
        return lines.stream().map(OrderLine::subtotal)
                    .reduce(Money.ZERO, Money::add);
    }
    public void place(Clock clock) {
        checkOpen();
        placedAt = Instant.now(clock);
        status = OrderStatus.PLACED;
    }
    protected void checkOpen() {
        if (status != OrderStatus.NEW)
            throw new IllegalStateException("order is " + status);
    }
}
```

Classes, a value object and an interface in Mermaid `classDiagram`




```
classDiagram
  direction LR
  class Product {
    -id: ProductId
    -name: String
    -price: Money
    +changePrice(p: Money) void
  }
  class Money {
    <<record>>
    -amount: BigDecimal
    -currency: Currency
    +add(o: Money) Money
    +times(qty: int) Money
  }
  class OrderRepository {
    <<interface>>
    +findById(id: OrderId) Optional~Order~
    +save(o: Order) Order
  }
  note for Money "value object: ..."
```

- Members: `visibility name: Type`; a member with `()` is an operation, the return type follows the parentheses.
- Generics use `~T~` because `<` and `>` are reserved for arrows.
- `<<interface>>`, `<<abstract>>`, `<<record>>`, `<<enumeration>>` are annotations on the first line of the body.

Visibility, types and modifiers: UML notation and its Java meaning

UML	Meaning	Java	Mermaid
`+ name`	public	`public`	`+`
`- name`	private	`private`	`-`
`# name`	protected	`protected`	`#`
`~ name`	package	no modifier (package-private)	`~`
`name: Type`	typed attribute	`Type name;`	`name: Type`
`name: Type [0..1]`	optional	`Optional<Type>` or nullable	text
`name: Type [*]`	many	`List<Type>` / `Set<Type>`	`List~Type~`
`name: Type = v`	default value	field initializer	text
name	static (class-level)	`static`	`name$`
`{readOnly}`	set once	`final`	text
`/name`	derived, computed	getter without field	text
italic op()	abstract	`abstract`	`op()*`

```java
public final class Order {
    // + MAX_LINES: int = 50  (underlined: static)
    public static final int MAX_LINES = 50;

    // - id: OrderId {readOnly}
    private final OrderId id;

    // - status: OrderStatus = NEW
    private OrderStatus status = OrderStatus.NEW;

    // - lines: OrderLine [*] {ordered}
    private final List<OrderLine> lines = new ArrayList<>();

    // + /total: Money   (derived: no field, computed)
    public Money total() {
        return lines.stream().map(OrderLine::subtotal)
                    .reduce(Money.ZERO, Money::add);
    }

    // # checkOpen(): void   (protected)
    protected void checkOpen() { /* ... */ }

    // ~ markPaid(p: Payment): void   (package-private)
    void markPaid(Payment p) { status = OrderStatus.PAID; }
}
```

💡 Java has no notation for `{readOnly}` on a collection: `final` only fixes the reference. Enforce it by exposing `List.copyOf(lines)`, never the field.

Responsibilities first, operations second



- A class is a bundle of **responsibilities**: things it knows (attributes) and things it does (operations). Write them as short sentences before drawing members (CRC cards work well).
- Each responsibility becomes one or more public operations with **intention-revealing names**: `place()`, not `setStatus(PLACED)`.
- Do **not** list `getX()`/`setX()` for every attribute; they are implied noise. Show accessors only when they are the design (for example a read-only view).
- Operations that only read state are marked `{query}`; Mermaid has no property syntax, so put it in a note or omit it.

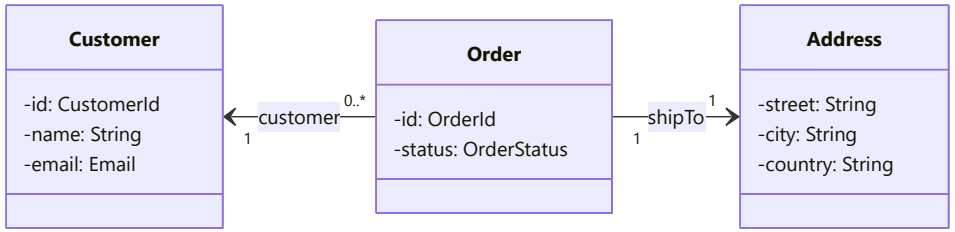
```
// Responsibility 3: enforce state transitions.
// The rule lives in Order, not in a service that pokes at setStatus().
public enum OrderStatus { NEW, PLACED, PAID, CANCELLED }

public final class Order {
    private OrderStatus status = OrderStatus.NEW;

    public void place(Clock clock) {
        checkOpen();
        if (lines.isEmpty())
            throw new IllegalStateException("empty order");
        placedAt = Instant.now(clock);
        status = OrderStatus.PLACED;
    }

    void markPaid(Payment p) {
        if (status != OrderStatus.PLACED)
            throw new IllegalStateException("not placed");
        if (!p.amount().equals(total()))
            throw new IllegalArgumentException("amount mismatch");
        status = OrderStatus.PAID;
    }
}
```

Associations: who knows whom, and how many




```
Customer "1" <-- "0..*" Order : customer
Order "1" --> "1" Address : shipTo
```

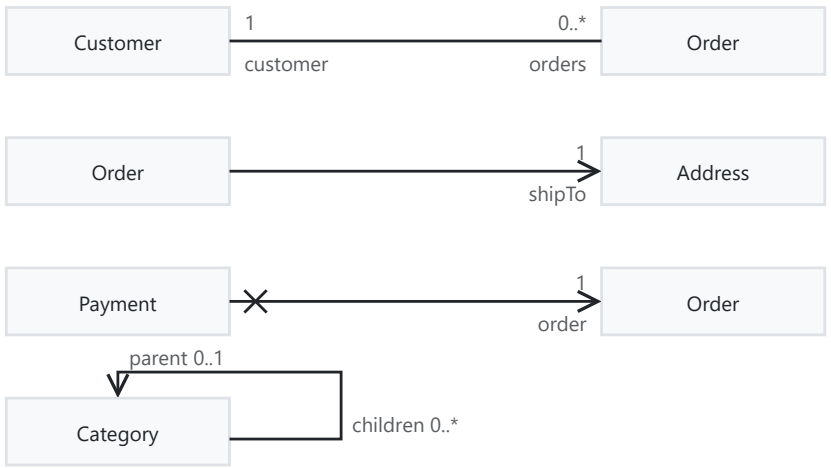
- An **association** is a solid line: instances of the two classes are linked at run time.
- **Multiplicity** at each end says how many instances take part: a Customer has `0..*` orders, every Order has exactly `1` customer.
- **Navigability** (open arrowhead) says which side holds the reference. Here `Order` knows its `Customer`; the Customer does not keep a list of orders.
- Mermaid puts the label after the colon; we use it for the **role name** at the arrow end (`customer`, `shipTo`). In a drawing tool, roles sit next to each end.

```java
public final class Order {
    private final OrderId id;
    // navigable Order -> Customer: the field lives here (role: customer)
    private final Customer customer;
    // role shipTo, multiplicity 1: never null
    private Address shipTo;

    public Order(OrderId id, Customer customer, Address shipTo) {
        this.id = id;
        this.customer = Objects.requireNonNull(customer);
        this.shipTo = Objects.requireNonNull(shipTo);
    }
}

public final class Customer {
    private final CustomerId id;
    private final Email email;
    // no List<Order> here: not navigable in this direction.
    // A customer's orders come from OrderRepository.findByCustomer(id).
}
```

Reading the line ends: arrows, crosses, roles, numbers



Bidirectional: both ends navigabl
no arrowheads at all (or both)

Unidirectional: Order knows Addr
Address has no reference back

× = explicitly not navigable
Order must never reference Paym

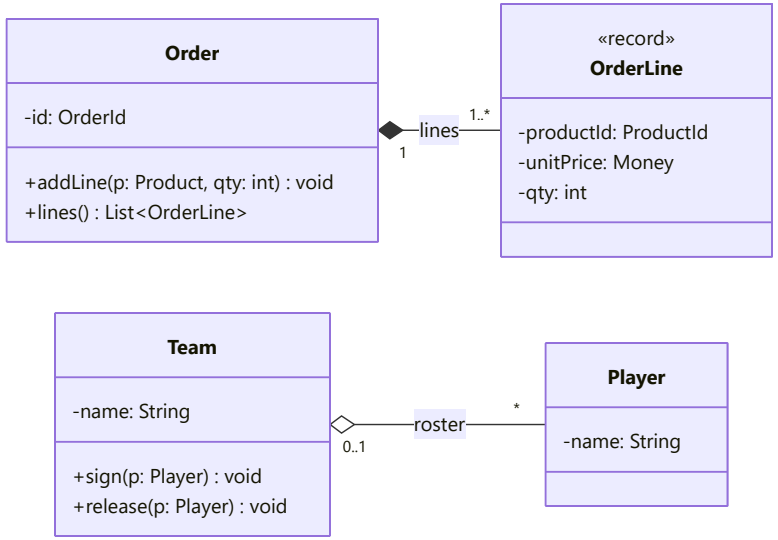
Reflexive: a class linked to itself
two roles distinguish the ends

Multiplicity	Reads as	Java field
`1`	exactly one	`final Address shipTo` (non-null)
`0..1`	optional	`Optional<Coupon>` or nullable
`*` / `0..*`	zero or more	`List<OrderLine>`, `Set<Tag>`
`1..*`	at least one	collection + guard in constructor
`2..4`	a range	collection + validation

```
// 1..* lines: the constructor enforces it
public Order(OrderId id, List<OrderLine> lines) {
    if (lines.isEmpty())
        throw new IllegalArgumentException("1..* lines");
    this.lines.addAll(lines);
}
```

⚠ An association with no multiplicity is unfinished. Decide it, or the implementer will.

Aggregation (◇) and composition (◆): whole-part relationships




```
Order "1" *-- "1..*" OrderLine : lines
Team "0..1" o-- "*" Player : roster
```

- **Composition** (filled diamond at the whole): the part belongs to exactly one whole, is created by it and dies with it. An `OrderLine` outside its `Order` is meaningless.
- **Aggregation** (hollow diamond): a looser whole-part grouping; the part exists independently and may be shared or moved. A `Player` survives the `Team`.
- The diamond sits at the **whole**; in Mermaid the whole is the left operand of `*--` / `o--`.

```java
public final class Order {
    // composition: lines are created, owned and destroyed by the Order
    private final List<OrderLine> lines = new ArrayList<>();

    public void addLine(Product p, int qty) {
        lines.add(new OrderLine(p.id(), p.price(), qty));
    }
    // defensive copy: nobody else can hold or mutate the parts
    public List<OrderLine> lines() { return List.copyOf(lines); }
}
public record OrderLine(ProductId productId, Money unitPrice, int qty) {
    Money subtotal() { return unitPrice.times(qty); }
}

public final class Team {
    // aggregation: players exist on their own and can be released
    private final Set<Player> roster = new HashSet<>();
    public void sign(Player p)    { roster.add(p); }
    public void release(Player p) { roster.remove(p); }
}
```

Choosing the diamond, and what it means for persistence

Question	Composition ◆	Aggregation ◇
Can the part exist without the whole?	no	yes
Can the part be shared by two wholes?	never	possibly
Who creates the part?	the whole	anyone
Deleting the whole deletes the part?	yes (cascade)	no
Typical Java	private list built inside, copies out	collection of references passed in
Typical JPA	`cascade = ALL, orphanRemoval = true`	`@ManyToMany` or plain `@OneToMany`

💡 When in doubt, draw a plain association. UML 2.5.1 leaves the precise semantics of aggregation to the modeller; composition has strict semantics.

```java
@Entity @Table(name = "orders")
public class Order {
    @Id private UUID id;

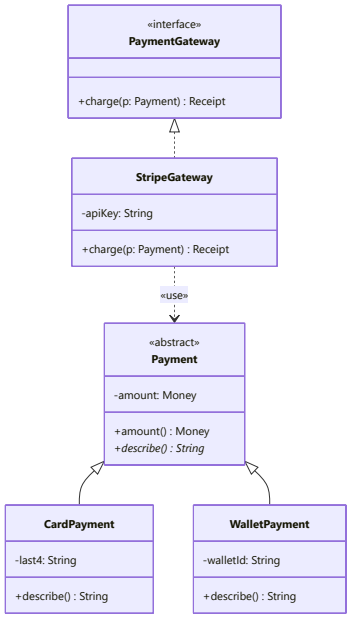
    // composition Order ◆-- OrderLine: lines share the Order's lifecycle
    @OneToMany(cascade = CascadeType.ALL, orphanRemoval = true)
    @JoinColumn(name = "order_id", nullable = false)
    @OrderColumn(name = "position")
    private List<OrderLine> lines = new ArrayList<>();

    public void removeLine(ProductId productId) {
        // orphanRemoval: the removed line row is deleted with the flush
        lines.removeIf(l -> l.productId().equals(productId));
    }
}

@Entity
public class Team {
    @Id private UUID id;

    // aggregation Team ◇-- Player: players are rows of their own,
    // deleting a team only deletes join-table rows, never players
    @ManyToMany
    @JoinTable(name = "team_roster")
    private Set<Player> roster = new HashSet<>();
}
```

Generalization (solid ▷) and realization (dashed ▷)




```
Payment <|-- CardPayment
Payment <|-- WalletPayment
PaymentGateway <|.. StripeGateway
```

- **Generalization**: solid line, hollow triangle at the *parent*. `CardPayment` is a `Payment`: it inherits attributes and operations and may override.
- **Realization**: dashed line, hollow triangle at the *interface*. `StripeGateway` promises to implement `PaymentGateway`.
- Abstract operations are italic (Mermaid: `describe()*`); the class itself carries `«abstract»` or an italic name.

```java
public sealed abstract class Payment
        permits CardPayment, WalletPayment {
    private final Money amount;
    protected Payment(Money amount) { this.amount = amount; }
    public Money amount() { return amount; }
    public abstract String describe();
}

public final class CardPayment extends Payment {
    private final String last4;
    public CardPayment(Money amount, String last4) {
        super(amount);
        this.last4 = last4;
    }
    @Override public String describe() { return "card **** " + last4; }
}

public interface PaymentGateway {
    Receipt charge(Payment p);
}

@Component
public final class StripeGateway implements PaymentGateway {
    private final String apiKey;
    StripeGateway(@Value("${stripe.key}") String apiKey) {
        this.apiKey = apiKey;
    }
    @Override public Receipt charge(Payment p) { /* HTTP call */ }
}
```

Closed hierarchies: `sealed` in Java, inheritance strategies in JPA

```java
// The same hierarchy as a sealed interface with records.
// In UML the arrows become realizations (dashed), the set stays closed.
public sealed interface Payment permits CardPayment, WalletPayment {
    Money amount();
}
public record CardPayment(Money amount, String last4)
        implements Payment {}
public record WalletPayment(Money amount, String walletId)
        implements Payment {}

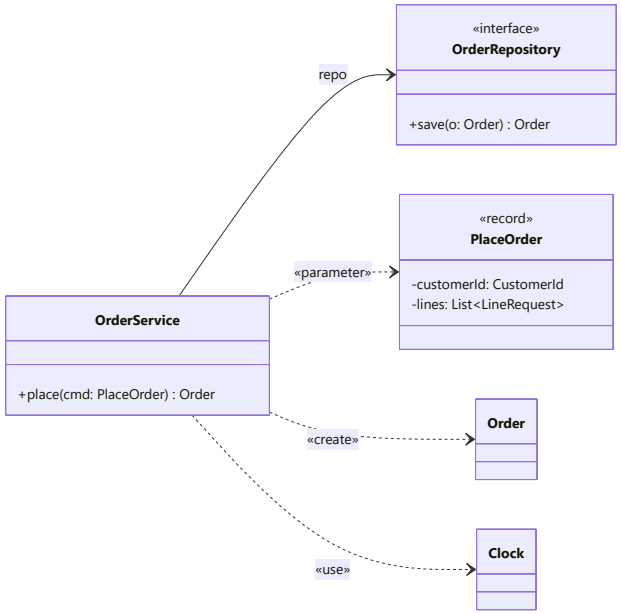
// Exhaustive switch: the compiler knows every permitted subtype,
// so no default branch is needed and a new subtype is a compile error.
static String describe(Payment p) {
    return switch (p) {
        case CardPayment c   -> "card **** " + c.last4();
        case WalletPayment w -> "wallet " + w.walletId();
    };
}
```

💡 Draw the generalization set as `{complete}` to say "no other subtypes exist": that is exactly what `sealed ... permits` enforces.

JPA strategy	Tables	Use when
`SINGLE_TABLE` (default)	one table, discriminator column, nullable subtype columns	few subtype-specific fields, fast polymorphic queries
`JOINED`	one table per class, joined by primary key	normalised schema, many subtype fields, NOT NULL constraints matter
`TABLE_PER_CLASS`	one table per concrete class, no parent table	rarely; polymorphic queries need UNION
`@MappedSuperclass`	no table for the parent	shared fields only (`id`, `createdAt`), no polymorphic queries

```java
@Entity
@Inheritance(strategy = InheritanceType.JOINED)
public abstract class Payment {
    @Id private UUID id;
    @Embedded private Money amount;
}
@Entity
public class CardPayment extends Payment {
    private String last4;
}
@Entity
public class WalletPayment extends Payment {
    private String walletId;
}
```

Dependency (dashed arrow): "I need you to compile, but I do not hold you"




```
OrderService --> OrderRepository : repo
OrderService ..> PlaceOrder : «parameter»
OrderService ..> Order : «create»
OrderService ..> Clock : «use»
```

- A **dependency** is the weakest relationship: a change in the supplier may force a change in the client. Drawn as a dashed line with an open arrowhead *towards the supplier*.
- Typical sources: a **parameter** or return type, a **local variable**, an object the client **creates**, a **static** call.
- Standard keywords: `«use»`, `«call»`, `«create»`, `«instantiate»`, `«derive»`, `«trace»`, `«refine»`.
- A structural collaborator held in a field (`repo`) is an **association**. A utility such as `Clock` is usually shown as `«use»` even if injected: the diagram is about the domain structure.

```java
@Service
public class OrderService {
    // association: a long-lived structural collaborator
    private final OrderRepository repo;
    // «use»: a utility the service calls, not part of the domain structure
    private final Clock clock;

    public OrderService(OrderRepository repo, Clock clock) {
        this.repo = repo;
        this.clock = clock;
    }

    // «parameter» PlaceOrder · «create» Order
    public Order place(PlaceOrder cmd) {
        var order = new Order(OrderId.next(), cmd.customerId());
        for (var l : cmd.lines()) order.addLine(l.product(), l.qty());
        order.place(clock);
        return repo.save(order);
    }
}
```

Dependency or association? Read it off the Java

In the Java code	In the diagram
field of a domain or collaborator type	association (solid, arrow to the field's type)
field of a value-object type (`Money`, `Email`)	attribute in the compartment, no line
collection field created inside the class	composition ◆
method parameter or return type	dependency `«parameter»` / `«use»`
local variable, `new X()`	dependency `«create»`
static method call `X.of(...)`	dependency `«call»`
`extends`	generalization
`implements`	realization

⚠ Never show the same field twice: either as an attribute *or* as an association line.

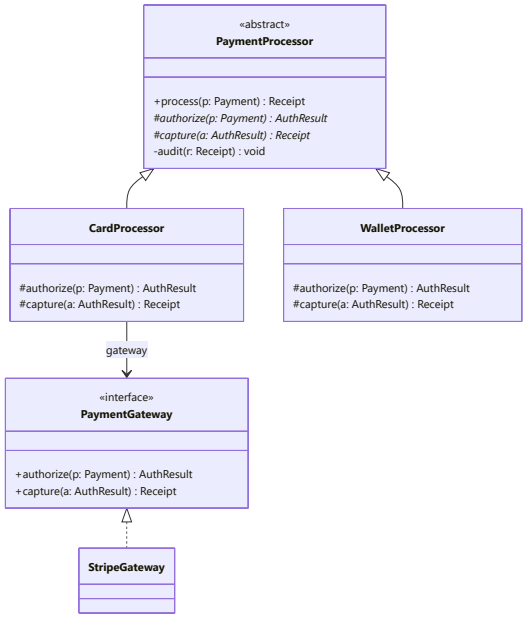
```
public final class Order {
    private final Customer customer;   // association  Order --> Customer
    private Money discount;            // attribute    - discount: Money
    private final List<OrderLine> lines = new ArrayList<>();
                                       // composition  Order *-- OrderLine

    // parameter: dependency  Order ..> Clock  «use»
    public void place(Clock clock) {
        checkOpen();
        this.placedAt = Instant.now(clock);
        this.status = OrderStatus.PLACED;
    }

    // local variable + static call: dependency  Order ..> PricingPolicy
    public Money total() {
        PricingPolicy policy = PricingPolicy.standard();
        Money sum = lines.stream().map(OrderLine::subtotal)
                         .reduce(Money.ZERO, Money::add);
        return policy.apply(sum, discount);
    }

    // return type: dependency  Order ..> OrderSummary  «create»
    public OrderSummary summary() {
        return new OrderSummary(id, lines.size(), total());
    }
}
```

Abstract class with a template method, interface behind it



- An **abstract class** fixes an algorithm and shared state; subclasses fill in the italic (abstract) steps. This is the *Template Method* pattern: `process()` is final, `authorize()` and `capture()` vary.
- An **interface** is a pure contract with no state; the concrete processor *uses* it through an association and Spring injects the realization.

```
public abstract class PaymentProcessor {
    private static final Logger log =
        LoggerFactory.getLogger(PaymentProcessor.class);

    // template method: fixed order, variable steps
    public final Receipt process(Payment p) {
        AuthResult auth = authorize(p);
        Receipt receipt = capture(auth);
        audit(receipt);
        return receipt;
    }
    protected abstract AuthResult authorize(Payment p);
    protected abstract Receipt capture(AuthResult auth);
    private void audit(Receipt r) { log.info("captured {}", r.id()); }
}

public final class CardProcessor extends PaymentProcessor {
    private final PaymentGateway gateway;
    public CardProcessor(PaymentGateway gateway) { this.gateway = gateway; }

    @Override protected AuthResult authorize(Payment p) {
        return gateway.authorize(p);
    }
    @Override protected Receipt capture(AuthResult a) {
        return gateway.capture(a);
    }
}
```

TOPIC 7

Interface, abstract class or sealed type? A decision table

	Interface	Abstract class	Sealed type
Expresses	a capability, a contract ("can be charged")	an is-a with shared state and a partial algorithm	a closed set of alternatives
State	none (constants only)	fields, constructors	either
Multiple	a class may implement many	single inheritance	n/a
Behaviour	`default` and `static` methods	concrete + abstract methods	exhaustive `switch`
Extension	open: anyone may implement	open unless `sealed`	`permits` list, same module
UML	`«interface»`, realization	italic name / `«abstract»`, generalization	`{complete}` generalization set
Shop example	`PaymentGateway`, `OrderRepository`	`PaymentProcessor`	`Payment`

💡 Rule of thumb: interfaces at the boundaries of a package (ports), abstract classes only when several subclasses really share code.

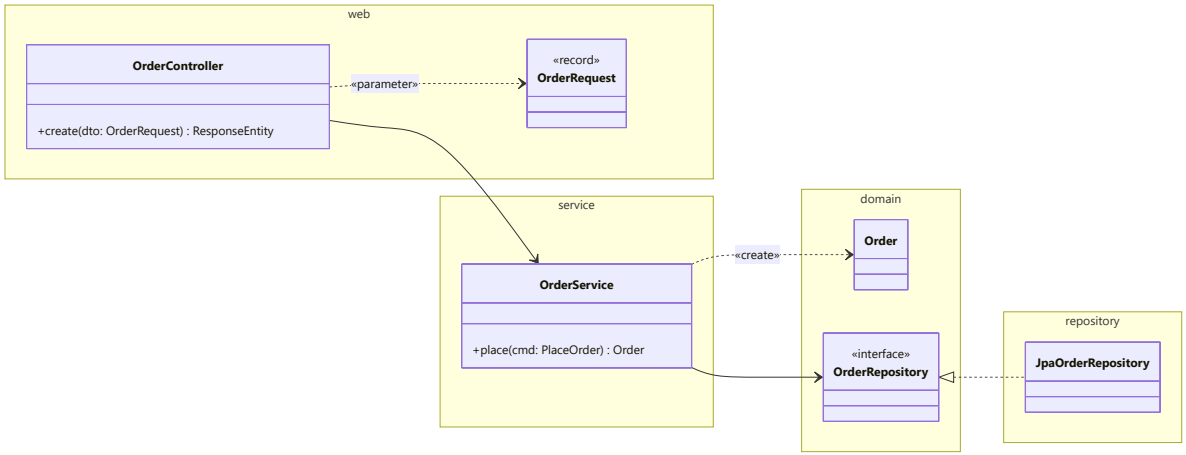
```
public interface PaymentGateway {
    AuthResult authorize(Payment p);
    Receipt capture(AuthResult auth);

    // default method: behaviour without state, overridable
    default boolean supports(Currency currency) { return true; }

    // static factory: useful in tests, no Spring needed
    static PaymentGateway accepting() {
        return new PaymentGateway() {
            public AuthResult authorize(Payment p) {
                return AuthResult.ok(p.amount());
            }
            public Receipt capture(AuthResult a) {
                return new Receipt(UUID.randomUUID(), a.amount());
            }
        };
    }
}

// generic interface: one contract, many aggregates (Ch. 03)
public interface Repository<T, ID> {
    Optional<T> findById(ID id);
    T save(T aggregate);
}
public interface OrderRepository extends Repository<Order, OrderId> {
    List<Order> findByCustomer(CustomerId customerId);
}
```

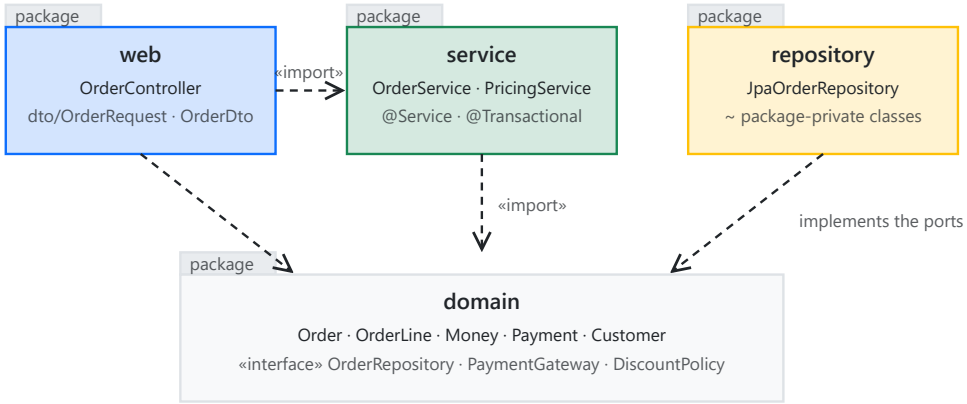
Packages group classes: Mermaid `namespace`




```
classDiagram
  direction LR
  namespace web {
    class OrderController {
      +create(dto: OrderRequest) ResponseEntity
    }
    class OrderRequest {
      <<record>>
    }
  }
  namespace service {
    class OrderService {
      +place(cmd: PlaceOrder) Order
    }
  }
  namespace domain {
    class Order
    class OrderRepository {
      <<interface>>
    }
  }
  namespace repository {
    class JpaOrderRepository
  }
  OrderController --> OrderService
  OrderService --> OrderRepository
  OrderRepository <|.. JpaOrderRepository
```

- A UML **package** is a namespace: `domain::Order` is the fully qualified name, exactly like `edu.itc.shop.domain.Order`.
- Relations are written outside the namespaces, between plain class names.

Package dependencies: arrows point inward, never in a cycle



Dependencies point towards domain; domain imports nothing from the other packages. service never sees repository.

```java
// domain: no Spring, no JPA imports. The model and its ports only.
package edu.itc.shop.domain;

public interface OrderRepository {
    Optional<Order> findById(OrderId id);
    Order save(Order order);
}

// repository: the adapter implements the domain's port.
// Package-private (~) class: only the Spring container sees it.
package edu.itc.shop.repository;

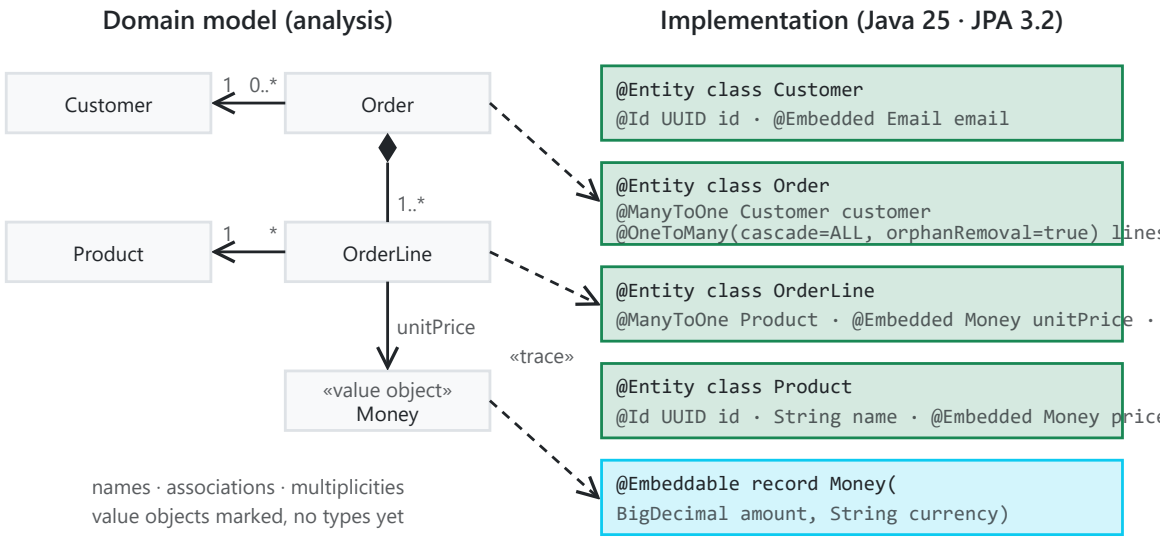
import edu.itc.shop.domain.Order;
import edu.itc.shop.domain.OrderId;
import edu.itc.shop.domain.OrderRepository;

@Repository
class JpaOrderRepository implements OrderRepository {
    private final SpringDataOrders orders;   // Spring Data interface
    JpaOrderRepository(SpringDataOrders orders) { this.orders = orders; }

    @Override public Optional<Order> findById(OrderId id) {
        return orders.findById(id.value());
    }
    @Override public Order save(Order order) { return orders.save(order); }
}
```

⚠ A cycle (`service → repository → service`) means the two packages are really one. Break it with an interface in the package that is used, not the one that uses.

From the domain model to Java 25 and Jakarta Persistence 3.2



- The **domain model** stays conceptual: no `UUID`, no annotations, no getters. Its job is agreement on vocabulary.
- The **design/implementation model** adds identifiers, value objects as records, ports as interfaces, and the persistence mapping.
- Keep a `«trace»` between the two: every domain class maps to exactly one implementation type (or you document why not).
- Deviations forced by JPA (no `final` entity class, protected no-arg constructor, mutable collection field) are recorded in the design diagram, not hidden.

```java
// Money is a value object: a record, embedded
// into any entity that has a price or an amount
@Embeddable
public record Money(BigDecimal amount, String currency) {
    public static final Money ZERO =
        new Money(BigDecimal.ZERO, "EUR");
    public Money add(Money o) {
        return new Money(amount.add(o.amount), currency);
    }
    public Money times(int qty) {
        return new Money(
            amount.multiply(BigDecimal.valueOf(qty)), currency);
    }
}
```

Translation table: UML element → Java construct → JPA annotation

UML	Java 25	Jakarta Persistence 3.2
class with identity	`public class Order`	`@Entity`, `@Id`
«value object» (Money, Address)	`record`	`@Embeddable` + `@Embedded`
attribute `- name: String`	`private String name`	`@Column` (implicit)
«enumeration»	`enum OrderStatus`	`@Enumerated(STRING)`
derived `/total`	computed method, no field	nothing, or `@Transient`
association → 1	`Customer customer`	`@ManyToOne(optional = false)`
association → 0..* (navigable)	`List<Order> orders`	`@OneToMany(mappedBy = "customer")`
bidirectional	fields on both sides	owning side + `mappedBy`
composition ◆	`final List` built inside, copies out	`@OneToMany(cascade = ALL, orphanRemoval = true)`
aggregation ◇	`Set<Player>` of references	`@ManyToMany` / `@OneToMany`, no `REMOVE`
generalization	`extends`, `sealed … permits`	`@Inheritance(strategy = JOINED)`
realization	`implements`	none (a Spring bean)
dependency	parameter, local, `new`	none
package	`package edu.itc.shop.domain`	none

```java
@Entity @Table(name = "orders")
public class Order {                          // not final: JPA proxies
    @Id private UUID id;

    @ManyToOne(optional = false)              // association → 1
    private Customer customer;

    @Embedded                                 // value object attribute
    private Address shipTo;

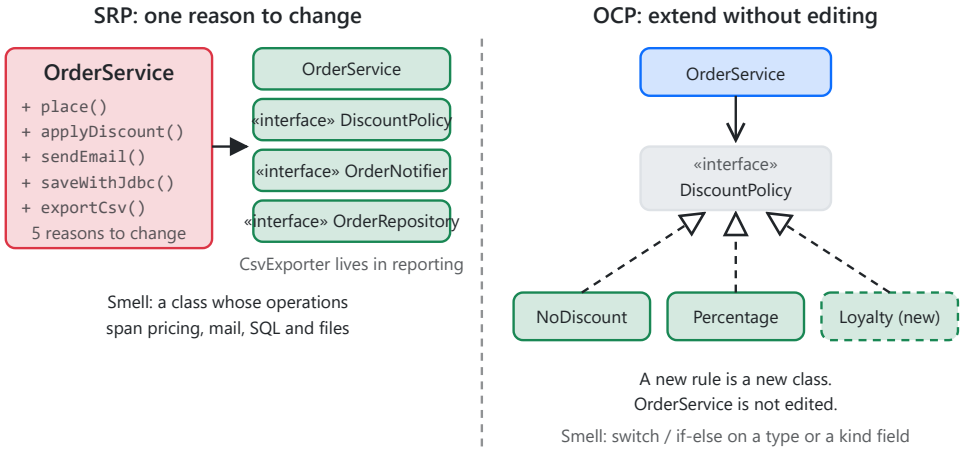
    @Enumerated(EnumType.STRING)              // «enumeration»
    private OrderStatus status = OrderStatus.NEW;

    @OneToMany(cascade = CascadeType.ALL, orphanRemoval = true)
    @JoinColumn(name = "order_id", nullable = false)
    @OrderColumn(name = "position")           // composition, {ordered}
    private List<OrderLine> lines = new ArrayList<>();

    protected Order() {}                      // JPA only
    public Order(UUID id, Customer customer, Address shipTo) {
        this.id = id; this.customer = customer; this.shipTo = shipTo;
    }
    public Money total() {                    // derived /total
        return lines.stream().map(OrderLine::subtotal)
                    .reduce(Money.ZERO, Money::add);
    }
    public List<OrderLine> lines() { return List.copyOf(lines); }
}

@Entity
public class OrderLine {
    @Id @GeneratedValue private Long id;
    @ManyToOne(optional = false) private Product product;   // → 1
    @Embedded private Money unitPrice;
    private int qty;
    Money subtotal() { return unitPrice.times(qty); }
}
```

SOLID in the diagram: Single Responsibility and Open/Closed




```java
// OCP: the variation point is an interface in the domain package
public interface DiscountPolicy {
    Money apply(Money subtotal, Customer customer);
}
public final class NoDiscount implements DiscountPolicy {
    public Money apply(Money s, Customer c) { return s; }
}
public final class LoyaltyDiscount implements DiscountPolicy {
    public Money apply(Money s, Customer c) {
        return c.isLoyal() ? s.percent(90) : s;
    }
}

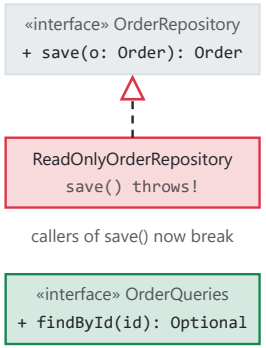
// SRP: OrderService only coordinates the use case
@Service
public class OrderService {
    private final OrderRepository repo;
    private final DiscountPolicy discount;   // pricing moved out
    private final OrderNotifier notifier;    // mail moved out

    public OrderService(OrderRepository repo, DiscountPolicy discount,
                        OrderNotifier notifier) {
        this.repo = repo; this.discount = discount; this.notifier = notifier;
    }

    public Order place(PlaceOrder cmd) {
        Order order = Order.from(cmd, discount);
        repo.save(order);
        notifier.orderPlaced(order);
        return order;
    }
}
```

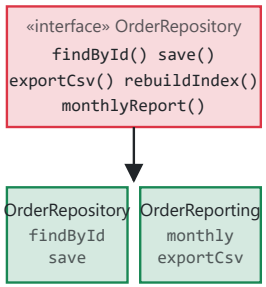
Liskov Substitution, Interface Segregation, Dependency Inversion

LSP: subtypes keep the contract



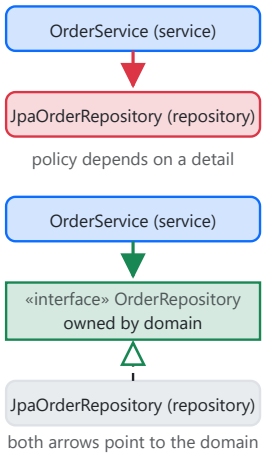
Fix: implement a narrower interface, no fake save()

ISP: no fat interfaces



Clients see only what they call:
OrderService → OrderRepository
ReportJob → OrderReporting

DIP: depend on abstractions




```java
// ISP: two small interfaces instead of one that does everything
public interface OrderRepository {
    Optional<Order> findById(OrderId id);
    Order save(Order order);
}
public interface OrderReporting {
    List<OrderSummary> monthly(YearMonth month);
    Path exportCsv(YearMonth month);
}

// DIP: the high-level policy depends on an abstraction it owns
@Service
public class OrderService {
    private final OrderRepository repo;   // never JpaOrderRepository
    public OrderService(OrderRepository repo) { this.repo = repo; }
}

// The detail implements the domain's interfaces (both of them is fine)
@Repository
class JpaOrderRepository implements OrderRepository, OrderReporting {
    /* Spring Data JPA behind it */
}

// LSP: a read-only variant realizes only what it can honour
public interface OrderQueries {
    Optional<Order> findById(OrderId id);
}
@Repository
class CachedOrderQueries implements OrderQueries {
    /* no save(): nothing to throw UnsupportedOperationException from */
}
```

Reviewing a class diagram with SOLID: smell → principle → refactoring

Smell in the diagram	Principle	Refactoring (what the arrows become)
One class with 15+ operations from different vocabularies (pricing, mail, SQL)	**S**RP	Split by responsibility; the old class keeps an association to each new collaborator.
An operation whose body would be a `switch` on a `kind: String` attribute	**O**CP	Introduce an interface (strategy) with one realization per kind; the client depends on the interface.
A subclass whose overriding operation is marked "throws Unsupported" or weakens a precondition	**L**SP	Move the operation to a narrower interface or make the hierarchy a composition.
An interface realized by classes that leave half of it empty	**I**SP	Split the interface by client; each client's arrow points only to what it calls.
A `service` class with an association to a `repository` or `infrastructure` class	**D**IP	Add an interface in `domain`; service → interface, adapter ⋯▷ interface.
A concrete class named after a technology used inside the domain (`StripeGateway` in `Order`)	**D**IP	Domain gets `PaymentGateway`; Stripe becomes an adapter in `infrastructure`.

```java
// The end state: OrderService depends only on domain
// abstractions. Every arrow from service points to
// an «interface» in package domain.
@Service
public class OrderService {
    private final OrderRepository repo;
    private final DiscountPolicy discount;
    private final PaymentGateway gateway;
    private final OrderNotifier notifier;

    public OrderService(OrderRepository repo,
                        DiscountPolicy discount,
                        PaymentGateway gateway,
                        OrderNotifier notifier) {
        this.repo = repo;
        this.discount = discount;
        this.gateway = gateway;
        this.notifier = notifier;
    }
}

// In a test every port is a fake: no Spring, no DB
var service = new OrderService(
    new InMemoryOrders(), new NoDiscount(),
    PaymentGateway.accepting(), order -> {});
```

☑ A SOLID diagram is easy to spot: concrete classes at the edges, interfaces in the middle, arrows pointing inward.

REVIEW

Common mistakes in class diagrams

1 Triangle at the wrong end

The hollow triangle sits at the *parent* or the *interface*. `CardPayment --|> Payment`, never the reverse. The dependency arrowhead points at the *supplier*.

2 Missing multiplicity

A bare line leaves the key decision (one or many, optional or mandatory) to whoever writes the field. Put a multiplicity on every association end you care about.

3 Attribute that should be an association

`- customer: Customer` hidden in the compartment of `Order` hides a structural link. Classes in the diagram are connected by lines; only value types stay as attributes.

4 Getter and setter noise

Twelve `getX()`/`setX()` rows drown the three operations that matter. Accessors are implied; list behaviour (`place()`, `total()`).

5 Same field drawn twice

An attribute `- lines: List<OrderLine>` *and* a composition line to `OrderLine` say the same thing twice, and will drift apart. Pick one.

6 Composition to a shared part

A filled diamond from `Order` to `Product` claims that deleting an order deletes the product. Products are shared: plain association, multiplicity 1.

7 Mixing analysis and design

`@Entity` and `UUID` in a domain model, or a design diagram with no types at all. Decide the level of the diagram and keep it consistent.

8 Everything on one sheet

Forty classes and a hundred lines communicate nothing. One diagram per package or per use case; 5 to 12 classes each; hide members that are not the point.

9 Diagram and code disagree

The diagram says composition, the code exposes the mutable list; the diagram says `«interface»`, the code injects the JPA class. Review the code against the diagram in every merge request.

REVIEW

Best practices for class diagrams that get used

1 Diagram as text, next to the code

Keep the Mermaid or PlantUML source in `docs/` in the same repository. It is reviewed, diffed and versioned like code, and rendered by the wiki.

2 Start with the domain model

Nouns from the use cases become candidate classes; verbs become associations or operations. Agree on names with the stakeholders before adding types.

3 Show what carries a decision

Members and lines are there to communicate choices: an identifier type, a composition, a port interface. Hide the rest with a note "accessors omitted".

4 Interfaces at package borders

Every arrow that leaves `service` ends in an `«interface»` owned by `domain`. Adapters in `repository` and `infrastructure` realize them.

5 Value objects as records

`Money`, `Email`, `Address` are immutable records, drawn as attributes, mapped with `@Embeddable`. Identity-bearing classes are entities.

6 Closed sets are sealed

If the generalization set is `{complete}`, write `sealed … permits` and switch exhaustively. The compiler then enforces the diagram.

7 Composition means ownership in code

The whole creates the parts, exposes copies, and the JPA mapping cascades with `orphanRemoval`. If any of the three is missing, it is not a composition.

8 Review with SOLID

Before coding, run the smell table: fat classes, `kind` switches, fat interfaces, service → JPA arrows. Fixing a diagram costs minutes; fixing code costs days.

9 Keep it consistent with other views

Every lifeline in a sequence diagram (Ch. 10) and every swimlane owner in an activity diagram (Ch. 09) must be a class or interface that exists here.

Chapter quiz 10 questions

1. Which visibility symbol marks a package-private member in UML?

- #
- ~
- -
- *

2. How is a realization (a class implementing an interface) drawn?

- Solid line with a hollow triangle at the interface
- Dashed line with a hollow triangle at the interface
- Dashed line with an open arrowhead at the class
- Solid line with a filled diamond at the interface

3. A filled diamond at the Order end of the Order-OrderLine line means…

- OrderLine is optional
- Order and OrderLine are both abstract
- OrderLine is a part owned by Order and dies with it
- OrderLine may be shared by several orders

4. Which Java code matches the composition Order ◆-- OrderLine best?

- public List lines; set from outside
- private final List lines = new ArrayList<>(); created by addLine(), exposed via List.copyOf
- private OrderLine[] lines passed into the constructor and returned directly
- a static List shared by all orders

5. Customer "1" <-- "0..*" Order in Mermaid says that…

- every order has exactly one customer and Order holds the reference
- every customer has exactly one order
- Customer holds a list of orders
- orders and customers are unrelated

6. OrderService.place(PlaceOrder cmd) takes a PlaceOrder parameter and stores nothing. In the diagram this is…

- an association OrderService --> PlaceOrder
- a composition OrderService *-- PlaceOrder
- a dependency OrderService ..> PlaceOrder
- a generalization

7. In Mermaid classDiagram, how do you mark a class as an interface?

- interface PaymentGateway { }
- class PaymentGateway <> on the relation line
- the annotation <> as the first line inside the class body
- PaymentGateway : interface

8. Which JPA mapping corresponds to a composition of OrderLine inside Order?

- @ManyToMany with a join table
- @OneToMany(cascade = CascadeType.ALL, orphanRemoval = true)
- @ManyToOne(optional = true)
- @Transient

9. OrderService (package service) has an association to JpaOrderRepository (package repository). Which principle is violated?

- Single Responsibility
- Liskov Substitution
- Interface Segregation
- Dependency Inversion

10. Which of these is a common mistake in a class diagram?

- Writing a multiplicity at both association ends
- Listing getters and setters as the operations of every class
- Using <> for a Java interface
- Drawing packages as namespaces

WRAP-UP

Summary

A class box has three compartments: name, attributes (`vis name: Type [mult] = default`), operations (`vis name(params): Return`).

Visibility `+ - # ~` maps to public, private, protected, package-private; underline = static, italic = abstract, `{readOnly}` = final.

Associations carry multiplicity at each end and navigability arrows; the side with the arrowhead is the side that owns the field.

Composition ◆: the whole creates, owns and deletes the parts (`cascade = ALL, orphanRemoval = true`). Aggregation ◇: parts live on their own.

Generalization (solid ▷) is `extends`; realization (dashed ▷) is `implements`; a `{complete}` set is `sealed … permits`.

Dependency (dashed arrow) comes from parameters, locals, return types and `new`; it is the weakest relationship and should stay inside a package or point inward.

Packages (`namespace` in Mermaid) group classes; dependencies point towards `domain`, which imports nothing.

Domain model → implementation: value objects become records and `@Embeddable`, entities get `@Id`, relationships get their JPA annotation from the translation table.

SOLID is visible in the picture: small classes, strategy interfaces, no throwing subtypes, split interfaces, arrows from services ending in domain interfaces.

Open Lab-08: 5 tasks + 1 challenge

Next chapter: 09 · Activity Diagrams